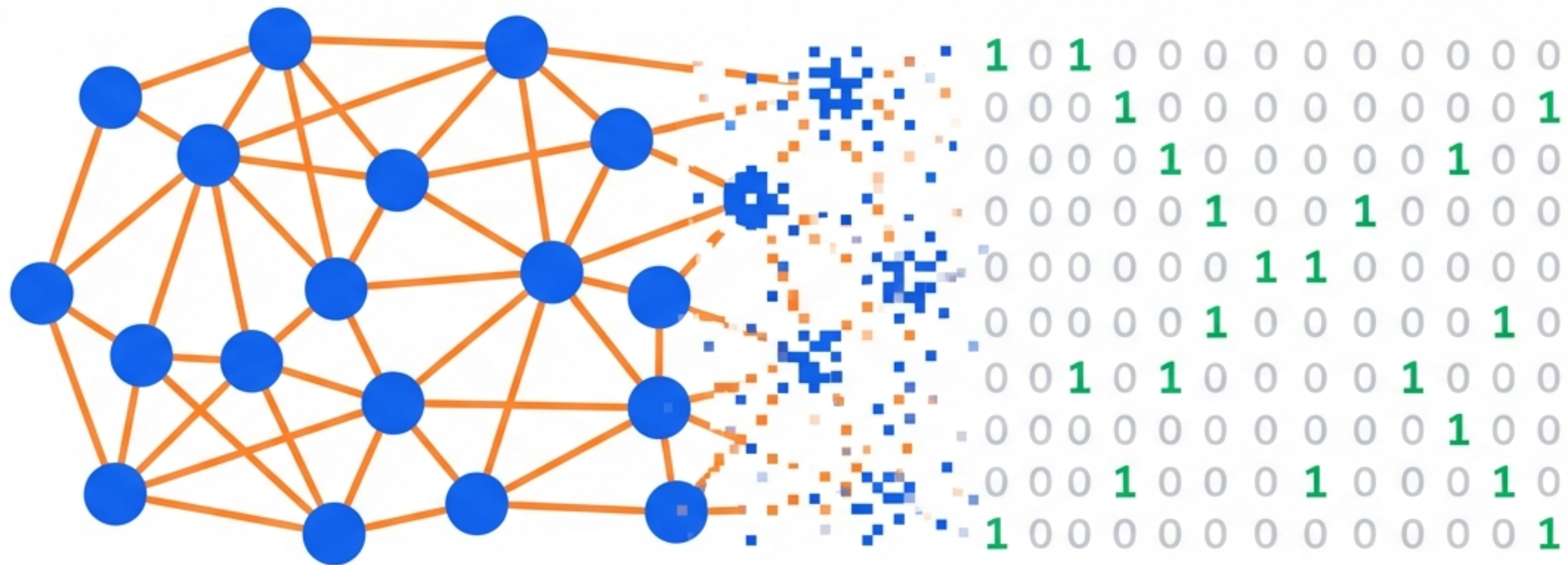
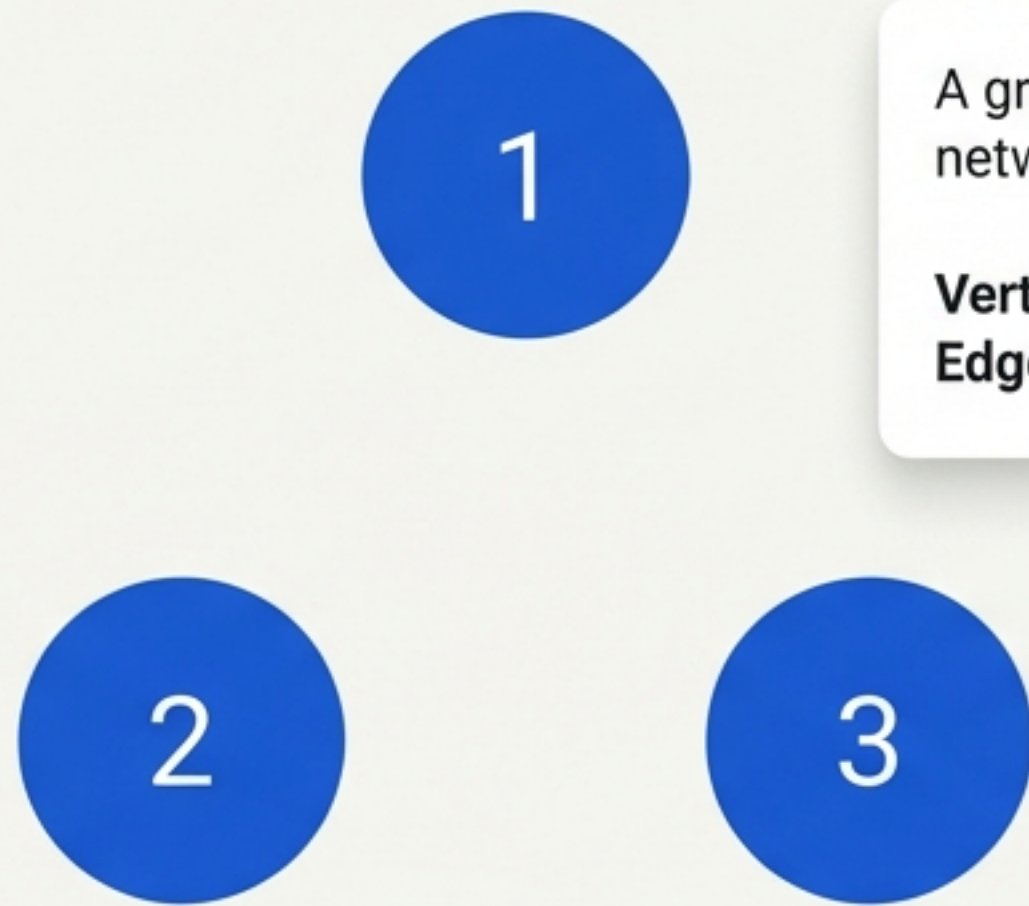


Graph Data Structures: Matrix Representation in Python



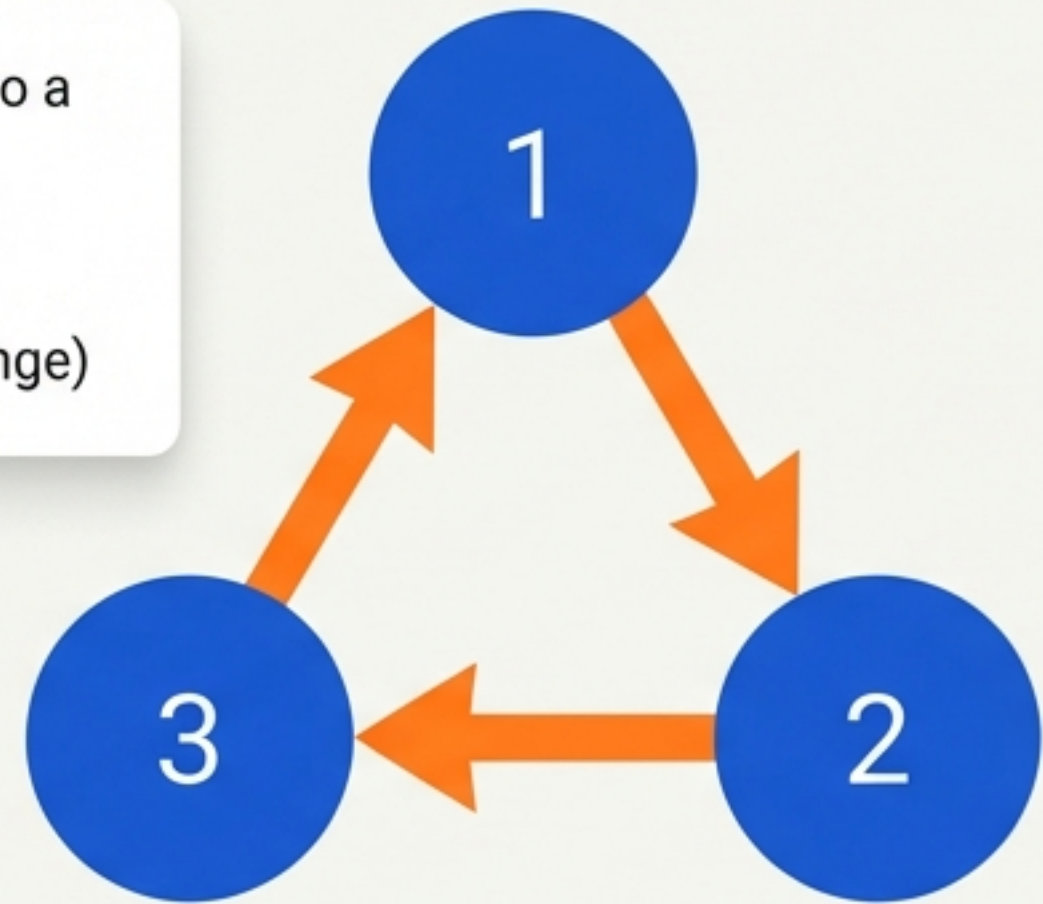
From Isolated Items to Ordered Pairs



Set A = {1, 2, 3}
Isolated Entities

A graph connects isolated points into a network.

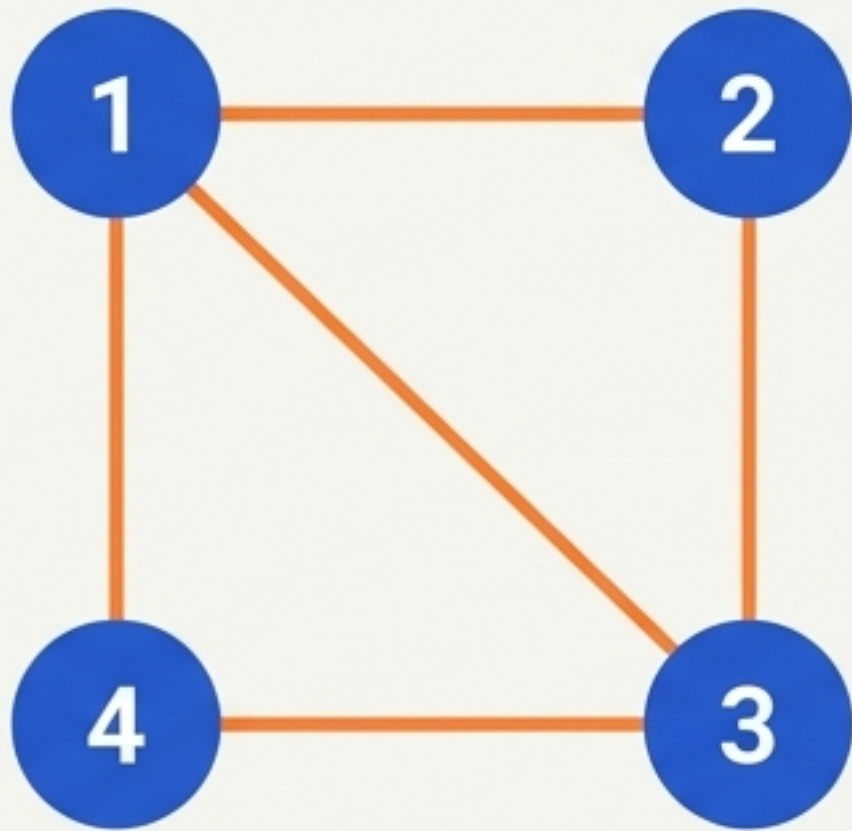
Vertices: The entities/nodes (Blue)
Edges: The connections/paths (Orange)



Set B = {(1,2), (2,3), (3,1)}
Ordered Pairs = A Graph

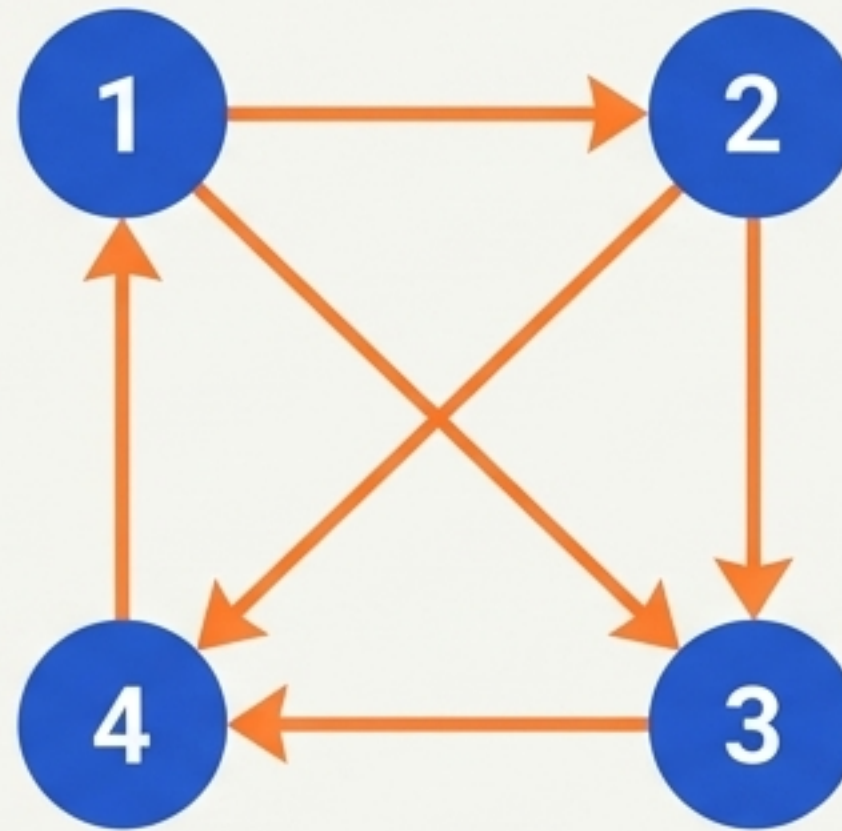
The Three Flavors of Graphs

Undirected Graph



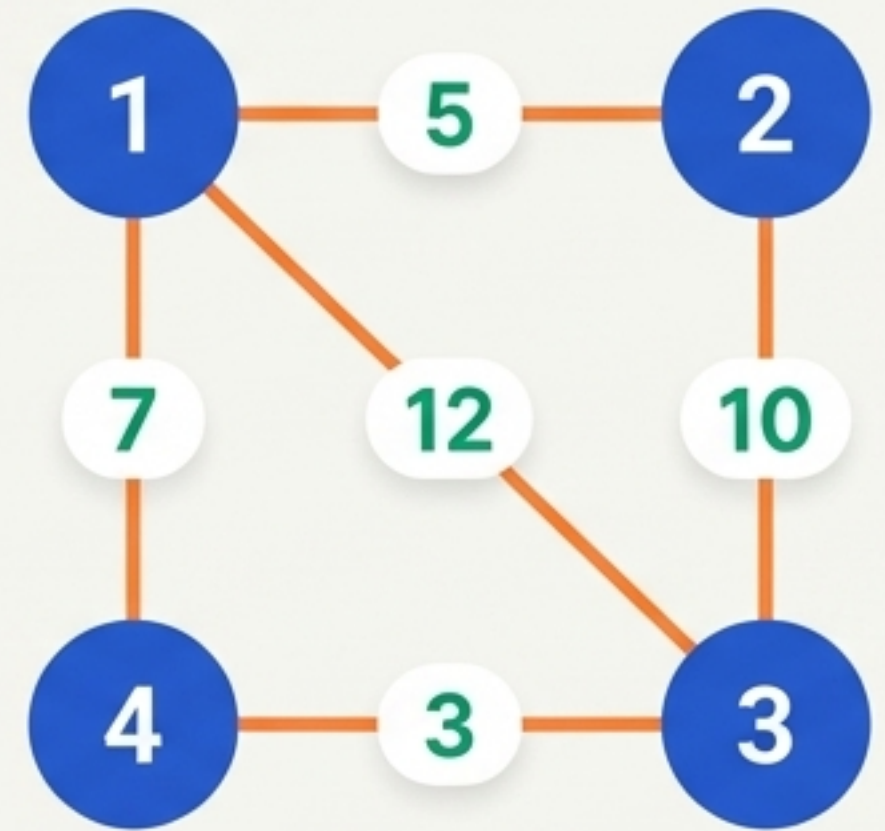
Two-way connections
(A to B implies B to A).

Directed Graph



One-way streets.
Direction matters.

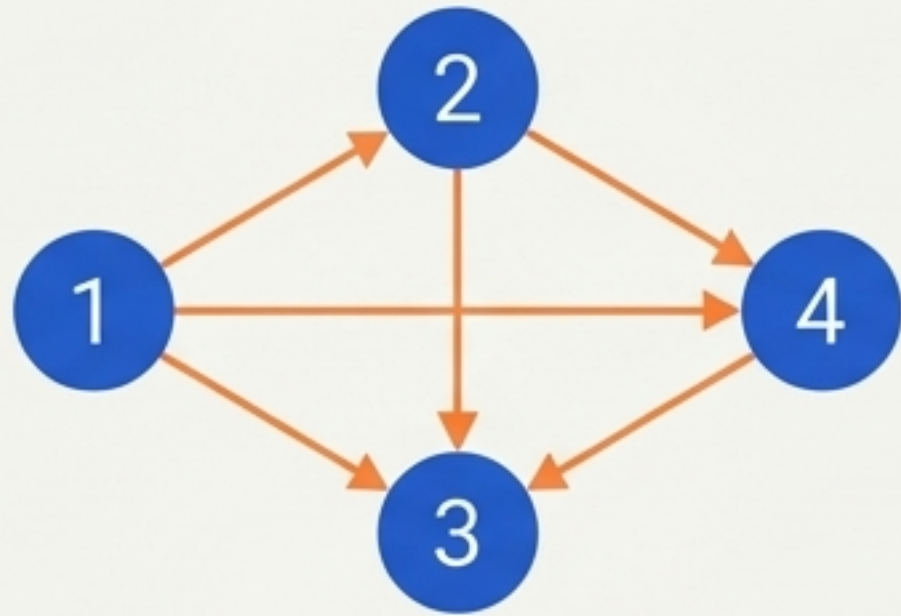
Weighted Graph



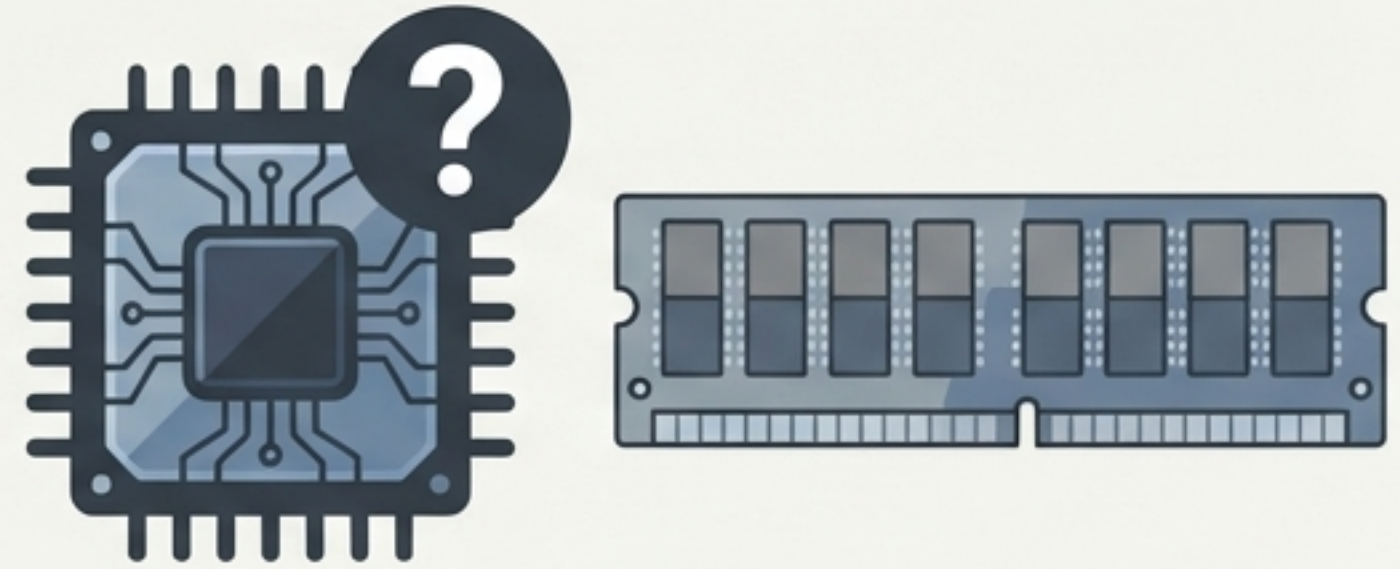
Edges carry specific
distances or costs.

The Computer's Dilemma

Human View



Computer View



How do we store this visual in memory?

Matrix Representation
(Focus of this deck)

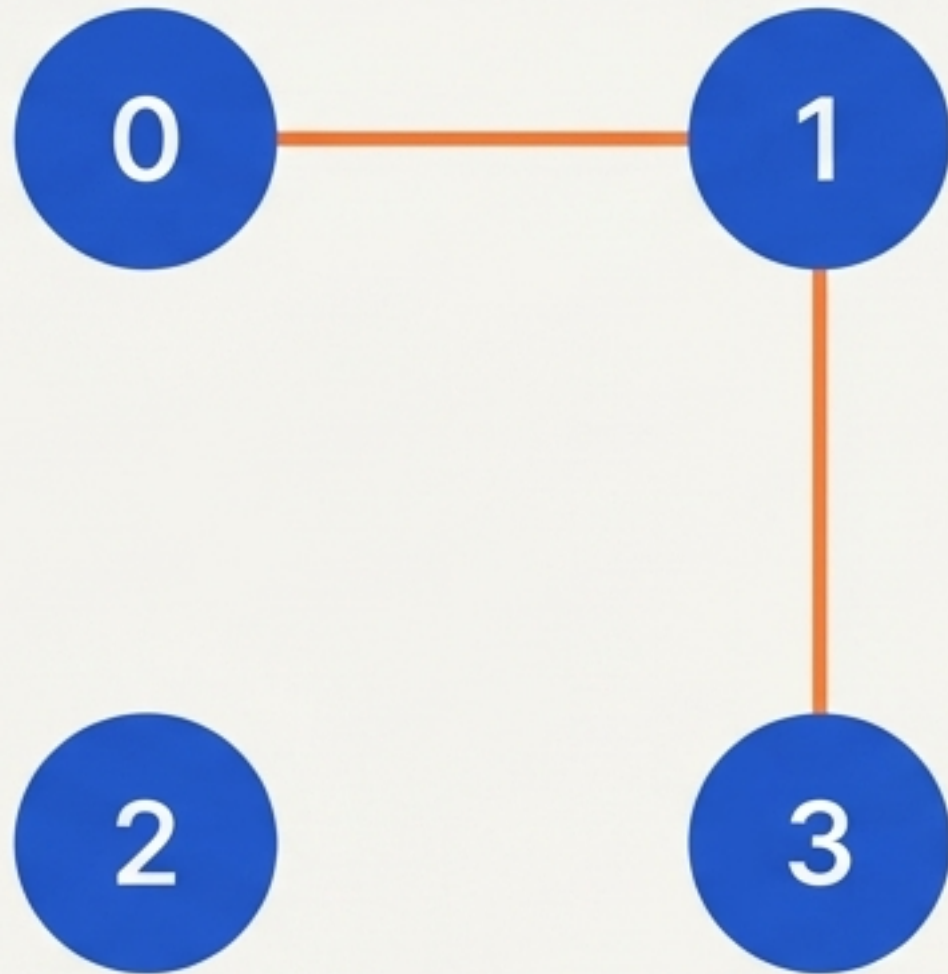
List Representation
(For sparse graphs)

Anatomy of an Adjacency Matrix

		Destination Node			
		0	1	2	3
Source Node	0	0	1	0	0
	1	0	0	1	0
	2	1	0	0	0
	3	0	0	0	1

- **V x V Grid:** The matrix size is strictly Vertices multiplied by Vertices (always a perfect square).
- **0** = No Edge (Path Absent).
- **1** = Edge Present (Path Active).

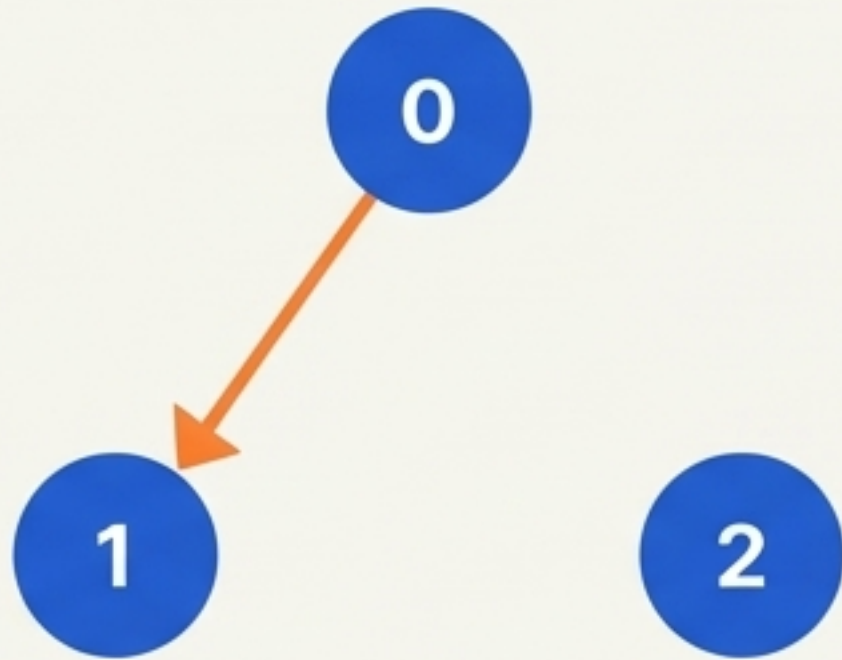
Visualizing the Matrix: Undirected Graphs



	0	1	2	3
0		1	0	0
1	1		1	0
2	0	1		0
3	0	0	0	

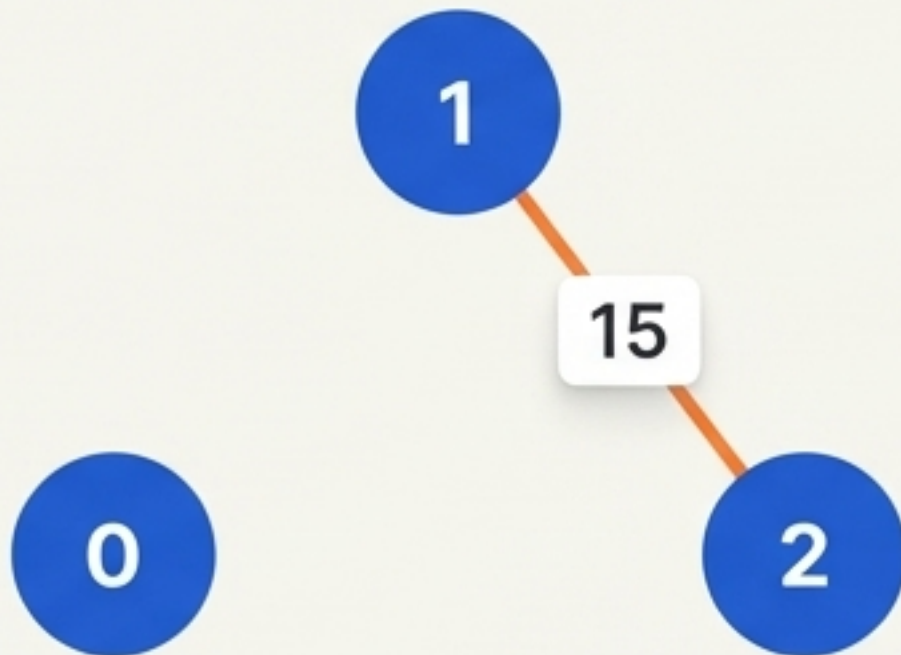
Symmetric Matrix: If an edge goes from 0 to 1, it must also go from 1 to 0. The green 1s perfectly mirror each other across the diagonal.

Visualizing the Matrix: Directed & Weighted



	0	1	2
0	0	1	0
1	0	0	0
2	0	0	0

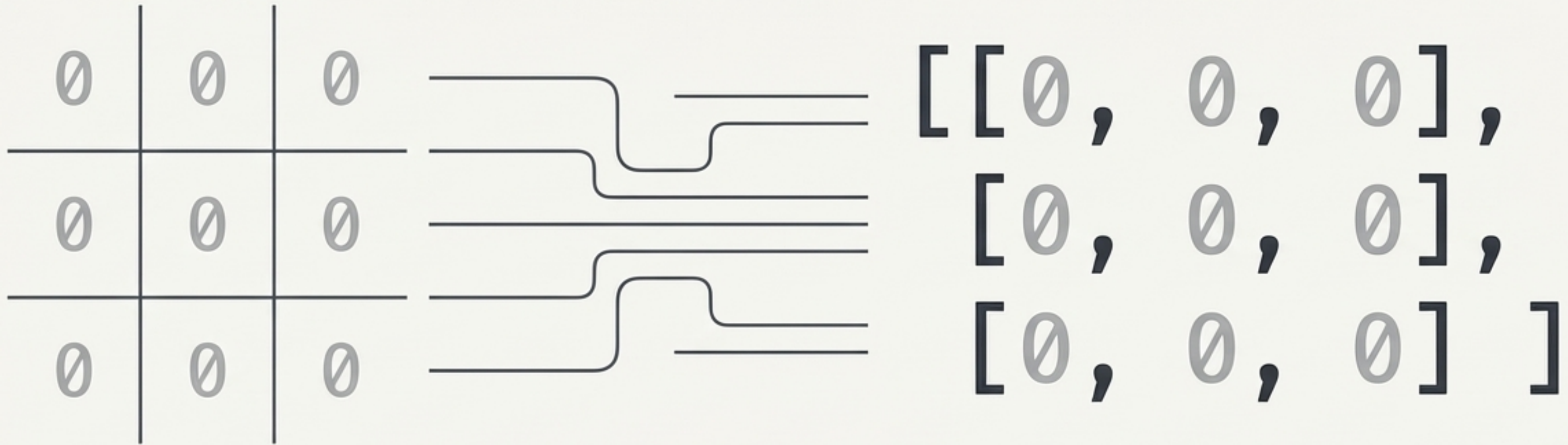
Directed Graph: No mirrored symmetry. 0 to 1 is active, but 1 to 0 is empty.



	0	1	2
0	0	0	0
1	0	0	15
2	0	15	0

Weighted Graph: Non-zero numbers replace the 1s to represent the edge weight or distance.

The Python Strategy: A List of Lists



Python doesn't have native 2D arrays. We simulate the matrix by nesting multiple lists inside one master list. Each inner list perfectly represents a single Row in the matrix.

Code: Initializing the Graph Grid

```
class Graph:  
    def __init__(self, vertex):  
        self.size = vertex  
        self.mat = [[0] * vertex for _ in range(vertex)]
```

Repetition Operator:
Creates a single row filled
with Light Gray zeros.

Generates exactly
enough rows to form our
perfect $V \times V$ square.

Code: Edge Logic & Boundary Safety

```
def add_edge(self, src, dest):  
    if 0 <= src < self.size and 0 <= dest < self.size:  
        # Add edge logic goes here  
    else:  
        print("Invalid Edge")
```



Index Out of Bounds Protection

This critical check prevents the program from crashing if a user tries to connect non-existent nodes (e.g., trying to access Node -1 or Node 5 in a 4-node graph).

Code: Assigning the Matrix Values

Directed / Weighted

```
self.mat[src][dest] = weight
```

Updates the single target destination only.

Undirected Graph

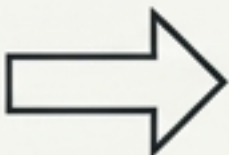
```
self.mat[src][dest] = weight  
self.mat[dest][src] = weight
```

Forces mathematical symmetry. Both directions are explicitly assigned.

Code: The One-Line Matrix Print

```
print(' '.join(map(str, row)))
```

1. `map(str, row)`

`[0, 1, 0]`  `['0', '1', '0']`

Converts integers to strings.

2. `' '.join(...)`

`'0'` `'1'` `'0'`

Combines the strings using a space separator.

3. `print(...)`

`0 1 0`

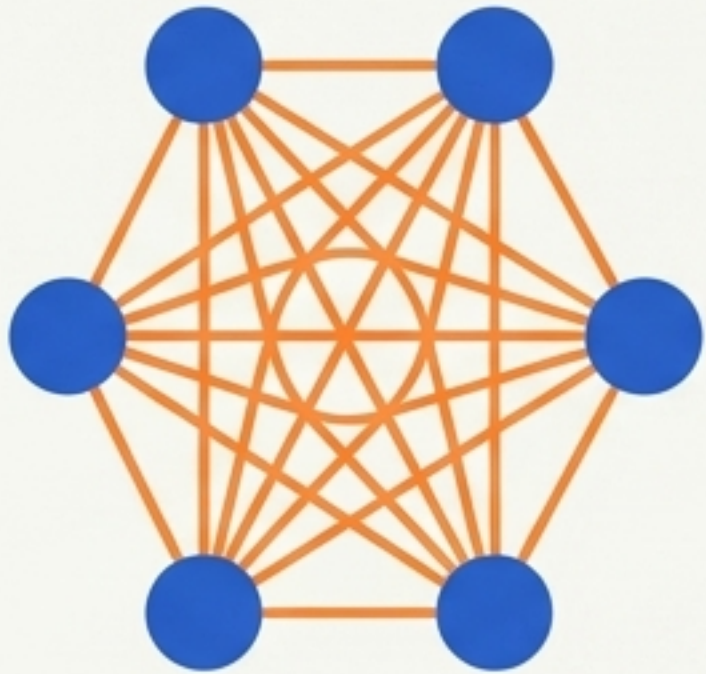
Displays the clean row output to the console.

The Verdict: Pros & Cons of the Matrix

Pros	Cons
 $O(1)$ Speed: Adding, removing, or checking the existence of an edge happens instantly by querying a specific index.  Simplicity: The Python implementation is incredibly clean, intuitive, and easy to debug.	 Memory Wastage: Even if a graph is completely disconnected, it still allocates a massive $V \times V$ grid of zeroes in memory.

The Golden Rule: When to Use a Matrix?

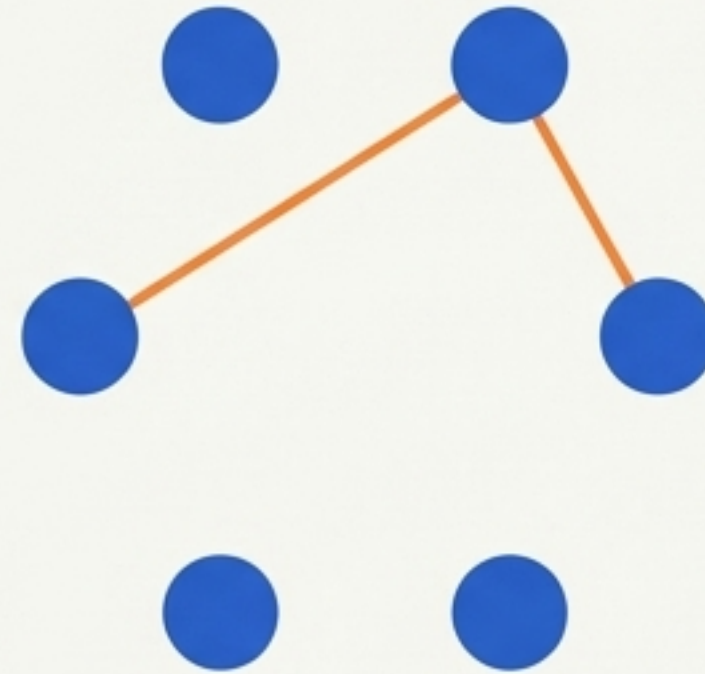
Dense Graph



	0	1	2	3	4	5
0	0	1	1	1	1	0
1	1	1	1	1	0	1
2	1	1	1	1	1	1
3	1	1	1	1	1	1
4	1	1	1	1	0	1
5	1	0	1	1	1	1

Verdict: Use a Matrix

Sparse Graph



	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	0	0	1	0
2	0	0	0	0	0	0
3	0	1	0	0	0	0
4	0	0	0	0	0	0
5	0	0	0	0	0	0

Verdict: Use a List (Save Memory)

Optimize for density. If the connections are few, the empty zeroes will silently consume your memory.